storygit: version control for story state

Arush Gumber
arush23136@iiitd.ac.in

Abstract

The hard part of long-form AI-assisted fiction is not generation but *state*: what is true, who knows it, what depends on it, and what a given edit therefore invalidates. storygit develops a one-line premise into a structured serial of episodes, scenes, beats and prose by modelling the story as a versioned typed state graph. Three mechanisms follow from that model. Every AI action is a reviewable diff against the graph rather than a block of replacement text. Edits propagate along declared dependency edges by *marking* what they affect and never by rewriting it. Continuity is checked graph-first, by a deterministic pass over a closed predicate vocabulary, before any model is asked for an opinion. A preference layer fits a Bradley-Terry ranking head to the accept, reject and edit signals a writer already produces, using their own stated criteria as features.

The evaluation is in two tiers. The deterministic tier needs no model calls and is therefore exact: the checker catches four of five injected contradiction classes at layer 1 and the fifth at layer 2, at 0.00 false positives per beat on a clean story, and declared dependency edges predict an edit's blast radius at precision 1.00 and recall 0.67. The live tier drives the real system with *simulated* writers, each a known weight vector over the head's own feature space, so what it can answer is whether the estimator recovers a taste it was shown, not whether the system helps a human write; no human study was run. That measurement is deconfounded twice: learning is read on a held-out probe of other writers' decisions, which cannot move because the task got harder, and recovery is scaled against an oracle ceiling computed with the same estimator on the same candidate sets at the same decision count, which separates what the learner did from what the sample size allowed. All development ran on free-tier inference at a total spend of $0.00.

1 Introduction

A writer starts with a sentence: *a powerless orphan discovers an ability that could change the balance of power in a world at war.* They want to end with a serial of tens or hundreds of episodes: coherent, in their voice, with the mechanics that make a listener press *next episode*. The naive form of AI assistance, prompt to story, fails at this in five reliable ways.

State drift. Facts established early are forgotten or contradicted later. The model has no ledger of what it has committed to, so episode 14 puts a character in a city they left in episode 9. The sharpest version of this failure is *epistemic*: a character acts on information the story has never given them. A reader forgives an inconsistent hair colour; they do not forgive a character who somehow already knows the secret.

Broken edit semantics. The writer changes one scene. Either nothing downstream responds, leaving a contradiction nobody flagged, or the system regenerates the plan and destroys work the writer had already approved. Both come from the same absence: nothing records what depends on what. Fabula's own study Mirowski et al. [2026] reports exactly this, from several writers independently.

Preprint. Under review.

Agency erosion. When the system proposes whole drafts, the writer's role collapses into judging them. Fabula's participants described becoming "an arbiter of good or bad" rather than an author Mirowski et al. [2026]. That is a worse job than writing, and it is the reason a tool can be impressive in a demo and unused in a month.

Exploration overwhelm. Offering more alternatives is not the same as offering more choice. Unlabeled variants cost more to read than they save, and a writer will stop opening them.

No learning. The thirtieth iteration repeats the first iteration's mistakes. Nothing in the system is a model of *this* writer's taste, so every correction has to be made again.

To these, serialized audio fiction adds a sixth: **no serial mechanics**. Hooks, cliffhangers, recaps, and the open threads a listener is waiting to see paid off are the retention surface of the product, and a general-purpose story generator does not track any of them.

Treat the story as a **versioned, typed state graph**. Make every AI action a **reviewable diff** against that graph rather than a block of replacement text. Make dependencies **explicit**, so that an edit propagates by *marking* what it affected and never by rewriting it. Let the writer **own the objective**, in their own words, rather than ranking against the system's. And **learn the writer's taste** from the signals they are already producing: what they accept, what they reject, and how they edit.

The division of labour follows: **the AI handles coherence and mechanics; the human handles taste, voice, and retention.**

Stated as a difference: Fabula is a chain of prompts with a user interface. storygit is a state machine with a learning loop. The chain is fast to build and pleasant to demonstrate; the state machine is what survives episode 40.

Contributions. Each mechanism below is derived from one commitment, that the story is a typed state graph under version control, and the evaluation is designed so the central claim can fail. The fourth is the one with no close precedent in this literature:

(1) A **versioned typed state graph** for a serial, with content-addressed snapshots, branches, and three-way merge, in which every AI action is a diff that is previewed, reviewed and committed rather than applied.
(2) **Deterministic dependency propagation with explicit staleness**: beats declare what they produce and consume, an edit walks those edges, and what it reaches is *marked*, never rewritten. Human prose is marked for review rather than staled.
(3) A **three-layer graph-first continuity checker**, in which a dictionary lookup over a closed predicate vocabulary runs before a cross-encoder, which runs before a model is asked anything, so the cheap and exact layer answers most questions.
(4) A **preference layer with a deconfounded measurement design**: a held-out cross-writer probe that cannot move because the task got harder, scored against an oracle ceiling computed with the same estimator on the same candidate sets at the same decision count.
(5) **Serialized-fiction mechanics** as first-class state: hooks, cliffhangers, recaps, and an open-thread ledger that reports how many beats each thread has gone untouched.

2 Related work

Fabula Mirowski et al. [2026] (DeepMind, 2026) builds a two-level hierarchical plan (a story plan of scenes, then beat plans per scene) and then a script, using prompt-chaining orchestrators with structured output. It is editable at every level, with top-down propagation and partial bottom-up updates. It ships 26 orchestrator variants, an auto-rater over 63 narratology guidelines that argues before it rates (to reduce position bias) and correlates 0.83 with human preference after tuning, a generate–critique–select search loop, and suggestion diversity implemented as embedding cosine against past suggestions. It was evaluated with 42 writers, comprising 18 design interviews and 25 three-hour sessions, plus hundreds of testers. It is the closest published system to this problem, and the immediate predecessor of these questions; Dramatron Mirowski et al. [2023] and Wordcraft Yuan et al. [2022] are the hierarchical-generation and writer-in-the-loop lines it builds on, and the creativity-support literature Cherry and Latulipe [2014] supplies the vocabulary for judging such tools.

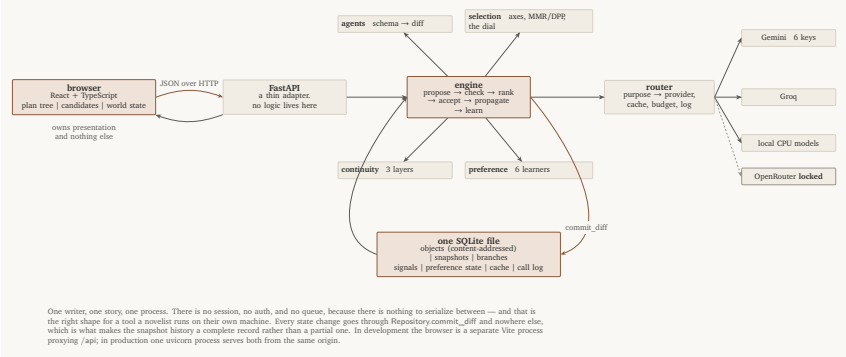


Figure 1: System components and the one path that writes state.

The study's findings matter more than its architecture, and they are unusually specific. Writers liked the structure, the scene breaking, the character questions, and the plan-beside-script view for restructuring. They disliked generic prose and poor handling of style, irony, and satire. They found the system *rigid*: they could not add a character locally, edits rewrote the whole plan, stale downstream scenes were not flagged. They asked for locks, insert and delete controls, progressive build-up, and an "absurdity dial". And the authors' own conclusion is that the system is most useful as feedback on the writer's own script, not as a generator.

Three of those complaints, namely edits rewriting the plan, stale scenes going unflagged, and the inability to add a character locally, are the same missing abstraction seen from three angles: there is no explicit dependency graph. storygit builds that abstraction first and derives the rest from it.

Gap	What storygit does about it
No dependency tracking; edits rewrite the plan	Beats declare produces / consumes; propagation *marks* affected nodes and never rewrites them
No per-character knowledge state	Epistemic edges knows(character, fact, since beat), checked before a character may act on something
No learning from writer signals	A preference layer trained on accept / reject / edit, with the writer's own criteria as features
No serial mechanics	Episode hooks, cliffhangers, recaps, and an open-thread ledger with last-touched dates
Diversity is "do not repeat the last one"	Axis-conditioned sampling with MMR or DPP selection over a quality-weighted similarity kernel

3 System

Five components and an interface. Everything below the interface is a pure Python library with no web machinery in it, which is why most of it is testable with no network and measurable with no server.

3.1 The state model

Five things are stored, and everything else is derived from them.

The plan tree. Story, Episode, Scene, Beat, Prose. Every node carries the four planning questions Fabula found writers actually use (what happens, what the audience learns, how they feel, where and when) plus a review status (draft, accepted, locked, stale) and, when stale, the reason. Episode nodes add the serial mechanics: hook, cliffhanger, recap of the previous episode, threads carried in and left out, writer-set goals, and a target length. Beat nodes add produces and consumes, the declared fact dependencies that make propagation exact.

The world graph. Entities (characters, places, objects, factions) with an alias table, and facts as edges between them. A fact carries a validity interval over the global beat order, so a fact that changes does not overwrite its own history: Kael is in Ashfall for beats 3 through 7 and elsewhere

afterwards, and both are retrievable at their own times. Predicates come from a closed vocabulary (location, alive, injury, possesses, relationship, relationship_status, goal, secret, trait, faction) plus a free-text note. Closed is what makes the first layer of continuity checking a dictionary lookup rather than a model call.

Using an explicit world-state graph to keep generated narrative consistent is an established idea Ammanabrolu et al. [2020]; what is added here is temporal validity, the epistemic layer, and the declared dependency edges that make edits propagate.

Epistemic edges. knows(character, fact, since beat), tracked separately from the facts themselves, because the difference between what is true and what a character has been told is the single richest source of continuity errors in generated fiction.

The open-thread ledger. Each thread records when it was opened and last advanced, so "you opened the sister subplot in episode 2 and have not touched it in nine episodes" is a query, not a judgement.

The writer ledger. Locks, hard constraints, style notes, writer-defined scoring criteria in the writer's own words, rejected directions, and the coherent-to-surprising dial. This is the object that keeps the human in authority, and everything in it is visible and deletable, including the rules the system mined from the writer's own edits, which must never influence generation invisibly. A hard constraint reaches the generation prompt *and* is checked at judge time: the judge receives the constraint list as explicit check items beside the scoring axes and returns the ones a candidate breaks, quoting each, which become soft flags. Never blocking, since the writer decides here as everywhere else, but checked, which is the difference between a constraint and a wish. Appendix B covers provenance, which is tracked per sentence rather than per node.

3.2 Snapshots and branches

Every object is stored once under the SHA-256 of its canonical JSON. A snapshot is a *manifest*: a mapping from logical id to content hash, plus a parent pointer, the author, and the intent line. A branch is a name pointing at a snapshot.

Three properties fall out for free. Versions are cheap: committing a change that touches three nodes writes three object rows and one manifest, so a five-hundred-snapshot history of a two-hundred-node story costs the changed nodes, not five hundred copies. Structural diff needs no heuristics: what changed between two versions is a set comparison over two manifests. And the whole thing is reproducible: identical content yields an identical manifest, so "apply the same diff twice and get the same object hashes" is a test rather than a hope. The snapshot *id* additionally hashes the timestamp, because a snapshot is an entry in a history and two identical edits an hour apart are two entries. With the clock pinned it too is content-derived, and the test asserts both halves.

Merging is three-way at object granularity. Where only one side changed an object, that side wins. Where both changed it differently, the merge returns a conflict. It never guesses which version of a scene the author meant.

3.3 The proposal engine

Every AI action is a **diff**: a list of typed operations against the current state, carrying a rationale and a delta summary in English. The writer reads *"introduces Mara; changes Kael's goal; would mark 2 downstream beats stale"* and decides. They never read a wall of replacement text and guess what it changed.

This is the direct answer to two of Fabula's findings at once. "I could not add a character locally" is, in this model, a single AddEntity operation. And "editing one scene rewrote the whole plan" cannot happen, because a diff that would rewrite the plan is visibly a diff that rewrites the plan.

Levels are proposed in order, and a level's schema requires the fields the level below it will need, so the writer settles the premise before episodes exist and episodes before beats. Appendix D gives the schemas and what each level's proposal must carry.

3.3.1 What the model is allowed to see

Never the story. Generation is fed an **entity-scoped slice**: the entities in scope, the facts about them *valid at that beat* with their ids, who knows what, the open threads, the plan path down to the insertion point, hard constraints from locks, the writer's style notes, and the writer's own criteria. A few hundred tokens instead of tens of thousands.

The important difference from ordinary retrieval-augmented generation is not the size, it is the kind of answer. Chunk-and-embed retrieval returns text that is *approximately relevant*; a query against a typed graph returns what is *exactly true, at this point in the story, about these entities*. That precision is what makes the continuity claims in §3.5 checkable rather than aspirational.

3.4 Propagation

Dependency edges are declared, not inferred: a beat states the facts it establishes and the facts it relies on, and an edge from beat A to beat B exists exactly when B consumes something A produced. When an accepted change alters or removes a fact, the system walks the consumers transitively and *marks* them. It never rewrites them.

The walk is exact over the edges that exist, and those edges are declared by the model into fields that are prompted for rather than schema-required. A beat that omits its dependencies validates, and a consumed id naming nothing is dropped without a flag, so the graph is as good as the model's compliance and this system does not measure that compliance. That is the least-guarded assumption here and §6 says so; Appendix D has the schemas.

Four rules make this safe to leave running.

1. **Locked nodes are never marked**, and the walk does not pass through them. A locked beat is not going to change, so nothing it produces can change either. The operational meaning of a lock is broader than that: every fact a locked node establishes is exported into every subsequent prompt as a hard constraint.
2. **Human prose is flagged for review, not staled.** The system does not tell an author that their own sentences are out of date; it tells them something they relied on moved, and leaves the judgement where it belongs.
3. **Nothing is ever rewritten.** The writer chooses: regenerate (previewed as a diff, respecting locks and current facts, and replacing the node rather than adding a sibling beside it), edit by hand, or dismiss it as still working. Regenerating a whole stale subtree exists, as an explicit, confirmed, previewed action.
4. **And accepting is not one-way.** An accepted node can be revised or deleted, on the same terms as striking a fact: the blast radius is shown before the commit, propagation marks what depended on it, and nothing below it is regenerated. Locks refuse, because a lock has to mean what it says. This is here because a writer using the system said the irreversibility changed how they worked: they over-deliberated on every candidate, because accept could not be taken back. A tool whose main verb is one-way makes its user slow, which is the failure mode this whole design is arguing against.

Because the marks are emitted as ordinary status operations, staleness flows through the same snapshot machinery as every other change. There is no side channel, and the history shows why each node was marked and by which upstream fact.

The same walk, run against a candidate that has not been accepted, is what produces the line the writer reads under every proposal: *"would mark 2 downstream beats stale"*. The blast radius is visible before the decision, not after it.

3.5 Continuity checking

Three layers, cheapest and most certain first. Each check is pushed to the cheapest layer that can decide it, and each layer's recall is reported separately, which is what turns "it keeps the story consistent" from an assertion into a measurement.

Layer 1: deterministic. For each new fact, look up existing facts with the same subject and predicate, valid at that beat. With a closed vocabulary that is a dictionary lookup returning zero to three

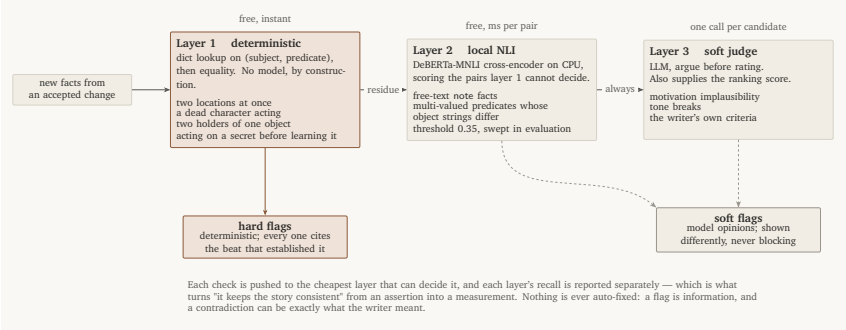


Figure 2: The three continuity layers, cheapest and most certain first.

facts, and the contradiction check is equality. It catches conflicting single-valued facts (a character in two places, alive and dead, in two factions), dead characters who act, two holders of one object, and, the important one, **epistemic violations**: a beat that relies on a fact, performed by a character who has not been told it. The flag says whether they learn it later ("learns it in *Beat D*") or never.

It deliberately does not flag a properly ended fact being replaced (a change, not a contradiction), a fact re-established with the same value (repetition), two goals at once (a character), or a character acting on their own secret.

Layers 2 and 3. The residue that layer 1 cannot type, chiefly free-text notes and prose, goes to a local natural-language-inference cross-encoder, and only what survives both reaches a model asked for an opinion. The ordering is the design: the cheap exact layer answers most questions, the expensive uncertain one is asked least often, and every flag carries which layer raised it so the writer knows what kind of claim it is. Appendix E gives both layers, the threshold, and what a flag must carry.

3.6 Diversity, and the dial

Six candidates that are near-paraphrases of each other are one candidate with a wider error bar. Diversity is therefore produced rather than filtered for: each sample is conditioned on a *named* axis, so the writer receives options labelled *raise the stakes*, *slow down*, *introduce a complication* rather than three unlabelled paragraphs, and the shortlist is selected by maximal marginal relevance or by a determinantal point process over a quality-scaled similarity kernel. A single coherent-to-surprising dial re-ranks the same sample by distance from the model's own temperature-zero continuation, which is a smaller claim than the label implies and is stated as such. Appendix G gives the selectors and the kernel. The λ default was chosen from a sweep whose objective was never written down, which is the weakest methodological point in this document and is recorded as such rather than defended.

3.7 The preference layer

Iteration thirty should not repeat iteration one's mistakes. Six learners over the same signal, namely accept, reject and edit, feed one ranking.

Why this layer exists, in one measurement. A writer ran a two-episode session through this system, and out of roughly twenty selections took the top-ranked candidate about four times. The best joke in episode one, hiring your own physiotherapist at £45 an hour to supervise a recovery you do not need, ranked *fifth of six*, at a quality score of 0.158. What ranked first, nearly every time, was the candidate that restated the writer's own instruction most literally.

That is the sharpest available statement of the problem. The judge is an LLM scoring a passage against axes, and what an LLM is reliably good at is recognising whether a text does what it was asked to do. *Compliance is what it can see.* A joke that works by going somewhere the instruction did not mention is, to a compliance grader, a candidate that drifted. So the ranker's first choice is systematically the least surprising member of the set, which is precisely the candidate a writer with taste is least likely to want.

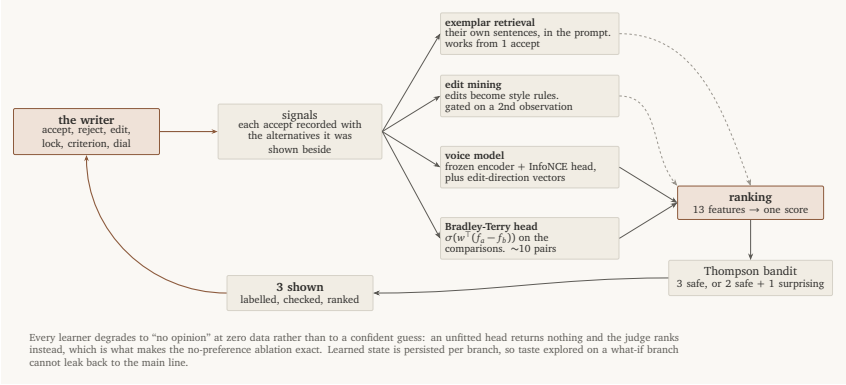


Figure 3: The preference loop: six learners over one signal.

Two things follow, and they are the whole design of this section. First, the shortlist is a shortlist and not an authority: all six candidates are shown, in full, and the ordering is a convenience rather than a verdict. Second, the ranking has to be able to move away from the judge, which is what the head is for. It consumes the judge's sub-scores as *features* rather than as a score, so a writer who keeps choosing fifth-ranked candidates teaches it to weight those axes differently. A system with only a judge cannot learn that its judge is wrong. That is the argument for everything below.

It is also an argument for a specific thing the fixed axes do not currently have: an axis that rewards a candidate for going somewhere the instruction did not specify. Every fixed axis here (momentum, specificity, consequence, voice) can be satisfied by a literal restatement. Adding one that cannot is the obvious next experiment, and it is not in this document because it has not been run.

Every accept is recorded with the alternatives that were on screen, so the unit of learning is a comparison rather than a rating; Appendix F sets out how pairs are formed and why they never cross shown sets.

3.7.1 The head

The head is logistic regression on feature differences, $P(a \succ b) = \sigma\big(w^\top (f_a - f_b)\big)$, with the intercept cancelling. Convex, milliseconds in NumPy, and the fitted w is directly readable, which is what lets the interface say *"you have been choosing for specificity and against length"*. It is the same construction as an RLHF reward model Bradley and Terry [1952], Christiano et al. [2017], Ouyang et al. [2022] minus the policy-gradient half; at one writer and a few dozen decisions, ordering candidates is the right ambition.

The feature vector is 13 numbers: four judge sub-scores on fixed narratology axes, four slots for the writer's own criteria, continuity, voice cosine, edit-direction projection, length, and dialogue ratio. Few and interpretable on purpose: a high-dimensional head would memorize twenty comparisons rather than learn from them. They are also the features the simulated writers' hidden weights are defined over, which is what makes §4's central question answerable.

The head starts from a prior fitted across synthetic proto-personas rather than from uniform weights, because twenty comparisons cannot fit a model from scratch. What that prior can and cannot transfer is in Appendix F; the cold-start row, where it does not help at all, is in the table in §4.

3.8 The interface

The interface is where the design either lands or does not. Fabula's finding that writers became "an arbiter of good or bad" is not fixed by a better model; it is fixed by changing what the writer is asked to do.

Diffs, not drafts. A writer reading a wall of replacement text is judging prose. A writer reading *"introduces Mara; changes Kael's goal; would mark 2 downstream beats stale"* is authorizing a change. Same underlying output, entirely different job.

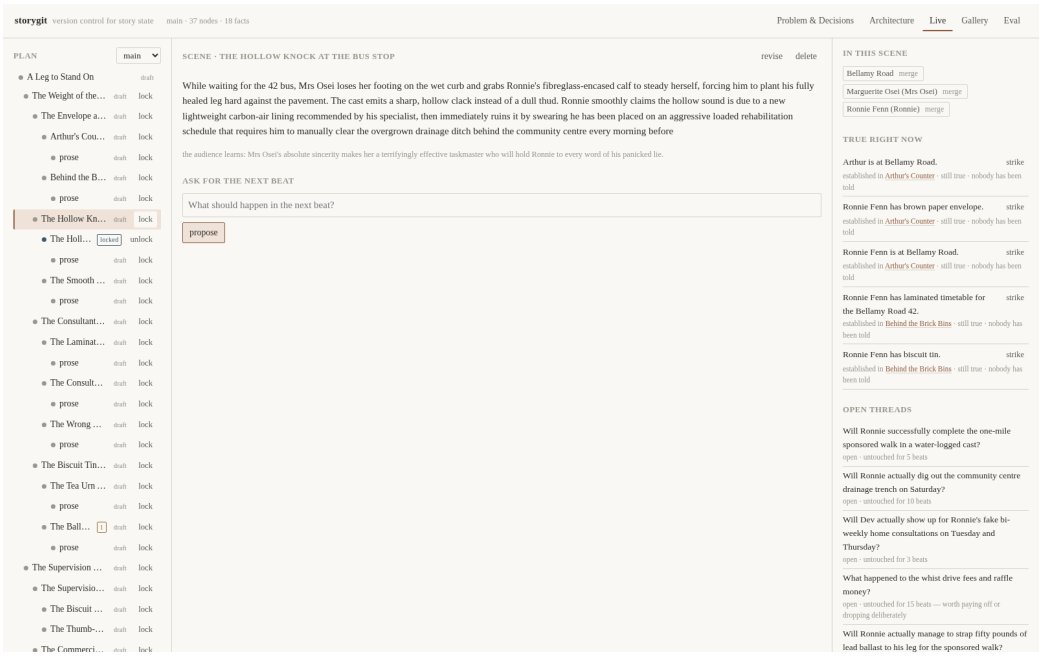


Figure 4: The Live tab over the demo story. Left: the plan tree, each node carrying its status as a word rather than only a colour, with its lock and its flag count. Centre: the selected node, with the revise and delete controls that make an accepted node changeable. Right: what is true at that beat, each fact citing the beat that established it and whether anyone has been told, above the open threads with how many beats each has gone untouched. Generating candidates needs provider keys; everything shown here reads from the committed story and needs none.

Named directions, not variants. Three options labelled *raise the stakes / slow down / introduce a complication* cost one decision. Three unlabelled paragraphs cost three readings, and a writer who has done that twice stops opening the third.

The blast radius before the decision. Every candidate says how many downstream nodes it would mark. That one line is the difference between a system a writer trusts with a manuscript and one they do not, because it makes the consequence visible while it is still avoidable.

Learning that is visible. Every rule mined from the writer's edits lands in the ledger, is shown, and can be deleted, and enters prompts only after being observed twice. A system that quietly adjusted its own prompts from inferred preferences would work slightly better and would be untrustworthy the first time it inferred wrong, and the writer would have no way to find out.

Nothing the system can do is API-only. A capability the writer cannot reach is a capability the product does not have, however well it is tested. The whole-graph audit, the three-way merge with its conflicts, the snapshot chain, striking a fact, merging a duplicate entity and deleting a mined rule are each a control in the tool, not just a route. This is checked rather than asserted: a test fails if any method in the API client is called by no component, and another fails if any state operation is constructed by nothing outside the two modules that define and apply it.

3.8.1 What the running system presents

The application is a five-tab site, and the tabs run in the order the story is told. **Problem and Decisions** states the six failure modes and the design decisions taken against them, each with the alternative that was rejected. **Architecture** shows the component diagram and the routing table. **Live** is the tool itself, in three panes: a plan tree carrying each node's status, its lock, its staleness reason and its flag count; a centre pane of labelled candidate diffs with their continuity flags, their blast radius and the coherent-to-surprising dial; and a right pane showing world state at the selected beat, including the epistemic view of who has been told what, the bible diff produced by the last accept, the branch list, the revise and delete controls on accepted nodes, and the whole-graph audit. **Gallery** replays eight recorded sessions from stored JSON, so the mechanisms can be watched

without a model call. **Eval** carries the curves and tables in this document, generated from the same artifacts.

Everything the backend can do is reachable from that interface, and a test fails if it is not. That is the difference between a capability the product has and a capability its test suite has.

4 Evaluation

Every claim in this document is either measured or explicitly marked as unmeasured. This section is the first kind; §6 is the second.

4.1 Results: the deterministic tier

These need no model calls, so they are exact: a regression is a regression rather than a bad sampling day, and they cannot be improved by rerunning.

Continuity checker	recall	precision	F1
layer 1 only (deterministic)	80%	100%	0.89
layer 1 + layer 2 (NLI)	100%	100%	1.00

0.00 false positives per beat on a clean story. That number matters more than recall: a checker that flags everything has perfect recall and is worthless, because the writer learns flags are noise and stops reading them. Layer 1 catches location conflicts, deaths followed by action, possession conflicts, and epistemic violations; the one class it cannot decide, two free-text notes that contradict each other, is exactly what layer 2 is for, which is the partition the closed vocabulary was designed to produce.

Staleness prediction	precision	recall	F1
declared edges only	1.00	0.67	0.80
best embedding-edge threshold	1.00	1.00	1.00

Declared edges are exact and blind to what nobody declared. The prediction stated in advance was that embedding edges would trade recall for precision too poorly to be useful; on this case the embedding edges reach 1.00 precision at 1.00 recall, and they do so over a *range*: 4 of the 6 swept thresholds tie there, from 0.68 to 0.85. That matters more than the peak. A single best threshold picked from the only case available is an argmax on the test set; a band over which the mechanism is indistinguishable from perfect is evidence that it is not balanced on one number. What ships is 0.72, inside that band. The case is six beats with lexically distinctive locations, so this is suggestive rather than settled. It is nonetheless the opposite of the prediction.

Selector	mean pairwise distance	mean quality
top-k (temperature only)	0.254	0.900
MMR ($\lambda = 0.5$)	0.483	0.833
DPP	0.555	0.780

Six candidates, of which the three highest-scoring are near-paraphrases of each other. The baseline takes all three. This measurement changed the system twice, described in §5.

Preference head, fresh writer	from prior	from uniform	lift
before any comparison	0.785	0.795	-0.010
after 5 comparisons	0.795	0.685	+0.110
after 10 comparisons	0.780	0.700	+0.080
after 20 comparisons	0.805	0.765	+0.040
after 40 comparisons	0.825	0.815	+0.010

Pairwise accuracy on comparisons held out from a proto-persona the prior was not fitted on. Two things about this table bound what it shows.

The cold-start row is negative: with no data at all the uniform head is the better ranker. The prior's advantage appears only once a small number of comparisons exists, and most of it is the *uniform* head being damaged by small-sample fitting rather than the prior being informative: the uniform column falls before it recovers, and a zero-data uniform head outscores the prior-initialised head at ten comparisons. The generator that produces proto-personas forces judge, criterion and continuity weights positive, which is the sign structure real personas have too, so an all-equal-positive vector is a decent ranker on this population before it is told anything. The prior is being compared against a baseline that starts strong.

And the held-out writer, while genuinely not one of the eight the prior was fitted on, is drawn from the same generator. This measures that the prior generalises within its own distribution. It is not a transfer test, and the four personas the live evaluation uses come from a distribution this generator cannot produce at all.

The bandit's numbers below are a simulator rather than the product: the arms are held at fixed rates and nothing in the running system composes a shown set through it, as §6 states. They measure the sampler, not the writer's experience. The Thompson sampler reaches cumulative pseudo-regret 4.8 over 400 rounds against arms at 0.40 and 0.70, and is flat after roughly the first fifty, because it pays its exploration cost early and then stops. ε-greedy at 0.1 reaches 6.3 and is still climbing linearly, because it explores at a fixed rate forever and explores uniformly. It spent 16 of 400 pulls on the safe arm against 384 on the better one, and its posteriors land at 0.389 and 0.707 against true rates of 0.40 and 0.70.

4.2 Results: the live tier

Four simulated writers drove the real engine through the full configuration with no human intervention: 4 runs, 14 writer decisions, mean acceptance 90%, and 42,584 tokens per decision. Of those, 0 ran to completion and 4 stopped early on provider rate limits, which is a property of running an evaluation on a free tier rather than of the system, and is reported here because a truncated run's later episodes are missing rather than bad.

The held-out probe. An acceptance curve cannot separate a head that learned from a task that got harder, so the learning result is measured on a frozen set instead. *[not measured yet]* decision points, sampled from *other* writers' runs, and from a sampling configuration that is never itself probed, are replayed after every episode: ranked by the head as it stood at that moment, scored against what that persona would privately have chosen. The probe set does not change between measurements, so nothing in it can move because the job got harder, and replaying it makes no model calls.

Which decisions go in is itself a design choice with a wrong answer. A candidate set where one option is better on every feature has the same winner under every weight vector, so no head can get it wrong and replaying it measures nothing, so those are dropped by a discrimination floor. But taking the *most* discriminating points instead, which an earlier version did, fills the set with near-ties: the decisions most sensitive to the weights and equally to noise, which maximises the variance of every reading. The rule is a floor and then a draw stratified across levels, and it never consults a head, fitted or otherwise.

Two details of the label are load-bearing. The persona's ranking is computed from the features as they were *recorded*, not as the current voice model would recompute them. Refreshing both sides let the ground truth drift between episodes, driven by the learner being measured, which is the leakage the held-out design exists to prevent. And the label reads only the nine features whose meaning is shared across writers: the criterion slots are positional, so one writer's first criterion is a different thing from another's, and every probe point comes from a different writer by construction. Scoring those columns across personas is not noise, it is a category error, and it lands on the highest-weighted feature each persona has. The head still learns and uses them; the held-out label does not read them.

A rank correlation means little on its own, so each reading carries two references scored over the same points at the same moment: a head that has learned nothing, and the population prior every writer starts from. Ending at the prior means the writer's own decisions have not yet added anything; ending above it means they have. ***[not measured yet]*** **of 4 runs end above an uninformed head,**

and *[not measured yet]* of 4 end at or above the prior, at *[not measured yet]* top-1 agreement with the persona's own first choice. The reading at this sample size is that the writers' own decisions mostly reproduce the prior rather than improve on it, and the mechanism behind that is identified below. The run that ends furthest below its references is *[not measured yet]*. Weight recovery, in the next paragraph, is the measurement that does move, and the two are consistent: the head is learning the directions its data exercises.

Weight recovery, against what it could have been. The correlation between the fitted weight vector and the persona's hidden one is 0.28 on average and 0.51 at best. That number is unreadable without a scale, and the scale is not 1.0: twenty-odd noisy comparisons over 13 features do not identify 13 weights. The *oracle ceiling*, which is the same estimator on the same candidate sets at the same decision count fitted to weights it is told, is 0.41 on average. Measured recovery is therefore 68% of what was achievable. Sample size, not the estimator, is what caps this: swept independently, the ceiling is 0.33 ± 0.22 at twenty-five decisions and still only 0.62 ± 0.16 at two hundred, because real candidate features are clustered and correlated and the design matrix is badly conditioned.

"The same estimator" is the clause this scale rests on, and it was not free. The first version of the ceiling was a different estimator in six ways: half the regularization, a uniform anchor instead of the population prior, no edited-win down-weighting, shrinkage off, a dense random weight vector where every persona is sparse by construction, and the persona's noise added to an *un-normalised* score, which made its labels several times cleaner than a real writer's. It also emitted a winner for every decision, including the ones the writer rejected, which produce no comparison at all in reality, so it was fitted on materially more data than the run it was scaling. Every one of those errors pushed the ceiling up and the reported fraction down, so the earlier figure was conservative rather than flattering. A sentence claiming a controlled comparison still has to have one. All six are now matched, and the ceiling is correspondingly lower.

Recovery is a 13-dimensional correlation, and five or six of every persona's 13 weights are exactly zero, including the two criterion slots no writer here fills, which are constant across every recorded decision. The shrinkage change drives the fitted values on precisely those dead coordinates towards zero, so it improves this metric without anyone learning anything. Restricted to the coordinates the writer actually weights, recovery is *[not measured yet]*. The difference between that and the headline is the size of the effect.

Two findings from the probe changed the learner, and one explains the writer who did worst. Both are set out in Appendix H, with the feature-level evidence; the summary is that several features never varied inside a single writer's candidate sets, that two of those were code defects and one was a property of the measuring instrument, and that the fix which shipped was decided by an offline replay against a rule written down before the number existed.

4.3 The strong-model rerun

Everything above ran on free-tier inference, which bounds prose quality in a way that has nothing to do with the design. The rerun exists to separate those two things. Generation and judging move to a metered provider on a stronger model; extraction and classification stay where they are, because moving them would change a second variable and cost money to learn nothing. The configuration is otherwise identical: same axes, same selector, same dial, same checker, same preference layer, same personas, same seed. The decision count is halved, because the full configuration on the cheapest model that fits does not fit the in-code budget cap, and a partial run of the full configuration would confound the comparison with wherever it happened to stop.

	free tier	**strong model**
runs	4	*[not measured yet]*
decisions	14	*[not measured yet]*
mean acceptance	90%	*[not measured yet]*
tokens per decision	42,584	*[not measured yet]*
weight recovery	0.28	*[not measured yet]*
probe tau, last episode	*[not measured yet]*	*[not measured yet]*
spend	$0.00	*[not measured yet]*

The spend figure is the one to read alongside the free-tier claim: *[not measured yet]* per writer decision, under a hard in-code cap of $12.00 that refuses a call it cannot afford rather than noticing afterwards. The cap was not raised for this run.

5 What the measurements changed

Two things the design predicted turned out wrong, and both changed the system.

MMR at the conventional $\lambda = 0.7$ was identical to the top-k baseline. Bi-encoder cosine has a high floor: six candidates here span 0.33 to 0.82, and any two English sentences on a related topic score well above zero. A redundancy penalty that is nearly constant across candidates subtracts nearly the same amount from every score, and a constant changes no ordering. At $\lambda = 0.7$ MMR selected at diversity 0.254 and quality 0.900, exactly the top-k baseline's 0.254 and 0.900. The headline selector was reproducing the ablation it was meant to beat. Fixed twice: the similarity matrix is rescaled within the candidate set, and the default λ moved to 0.5, chosen from a sweep.

The same measurement produced a better argument for DPPs than the one this document originally made. The DPP reaches the diverse answer *with no parameter at all*, because its kernel is multiplicative: a near-duplicate's contribution to the determinant collapses however good it is. MMR's additive penalty can always be outvoted by a large enough quality gap. That is a structural difference rather than a tuning difference.

Embedding-similarity dependency edges did better than predicted, as above.

Two runs ended on nothing. The first four-persona run stopped two writers at episode three, and reading the logs rather than the summary showed why: a persona rejects a candidate set when the best option falls below its own bar, and one of those two runs missed by **0.0001**, one ten-thousandth of a score unit, against a Gaussian noise term of the persona's own making. Nothing was degrading. Accumulated writer rules, the obvious suspect, turn out to be 0.4–1.2 % of a late prompt.

What was wrong was the harness's stop rule: it ended the session where a writer would have said *"not these, try again"*. So a rejected set now gets exactly one rejection-informed re-proposal, which also required wiring a path the system had described but never built: the rejected candidate's text reached the exemplar pool, while the writer's stated reason reached nothing. In the rerun the retry was offered 1 times and rescued 0 of them.

6 Limitations

The evaluation cannot measure taste. A persona is a linear functional over the same 13 features plus noise. It cannot be surprised, cannot change its mind, and cannot tell you the tool is exhausting to use. Every number here is a property of the machinery. Whether the system helps a human write a better serial is not established by anything in this document, and only a human study would establish it.

One writer did use the system, for a two-episode session, and this document quotes them three times: on accepting being irreversible, on hard constraints not binding, and on the ranker preferring compliance. Those are design evidence and are treated as such. They are one person's reaction, unblinded, uncontrolled and not a measurement, and no number anywhere here comes from them.

The preference layer is validated against writers built from its own hypothesis class. The head is linear in 13 features; the personas score linearly in the same 13. The model is therefore correctly specified by construction, which is why the claim is *recoverability*, meaning the estimator is not broken, rather than *it learns taste*. A real writer's preferences are not linear in 13 features.

The prior is synthetic. Nothing in the data it was fitted on ever met a reader. The bet is that the adaptation machinery transfers even though the values do not, and that bet is measured (a +0.080 lift at ten comparisons) rather than assumed.

Injected errors are the errors that were thought of. Checker recall is over five known classes. A class nobody imagined is a class nobody measured. The periodic audit exists partly for that. Recall is therefore over five classes.

Single writer, single story. SQLite, one process, no auth, no concurrency. The scaling discussion is architectural rather than demonstrated.

Fact extraction is a single point of failure. Everything downstream, meaning continuity, propagation and the world-state pane, depends on the world graph being populated correctly. Extraction recall is measured, but a fact that is never extracted is invisible to every layer that follows, and no amount of downstream cleverness recovers it.

Declaration is a second single point of failure, and it has no number. Extraction's failure mode at least has a measurement. The dependency edges the propagation walk reads are *declared* by the proposing model into optional fields, so a beat that quietly omits what it consumes produces no edge and no warning, and a consumed id that names nothing is dropped silently. The staleness figures above are measured on a dependency graph written by hand. Nothing here measures the graph a model actually produces, and it is the more likely of the two to fail.

Two of the learners are not on the writer's path. The Thompson bandit and the population prior are built, tested and measured, and they are exercised only by the evaluation harness: the product ranks with the selector's top k and starts a new writer at uniform weights. A third, smaller version of the same gap: the candidate *text* a comparison was fitted from is held in memory rather than persisted, so a restart leaves the head fitting on this session's pairs alone.

The instrument is coarser than the thing it measures. The judge emits five levels. Four of the 13 features and every writer-defined criterion come from it, and with three candidates in a set most of those features tie. In the shipped probe fixture the two criterion slots no persona fills are constant by construction, and several judge axes are constant in more than half the decisions. The effective dimensionality of a decision is far below 13. That is a property of the measuring instrument, not of the two feature bugs reported in Appendix H, and fixing those did not change it.

The probe's ground truth is not well defined across writers. The held-out probe scores each head against another writer's decisions, which is what deconfounds learning from task difficulty. But writer-defined criteria occupy positional slots, and two writers' first criteria are different things, so the cross-persona move and the per-writer criterion slots are in permanent tension. The probe resolves it by reading only the nine features whose meaning is shared. The criterion axes, which are the highest-weighted features for every persona, are therefore outside the number the learning curve reports.

The cost accounting is not complete, though the dollar figure is. Embeddings and the NLI checkpoint run locally and are called directly rather than through the router, so they produce no call-log row and no token count. Everything metered is free-tier; nothing unmetered is paid. But "every call is logged with its tokens" is true of the routed calls and not of all of them.

Nothing here carries an interval, and the ns are small. Four runs. Five injected errors. Three true positives in the staleness case. One sentence pair reaching layer 2. Six candidates in the selector case. The per-run ceiling spreads are computed and reported in the run summary; the figures in this document are point estimates, and several of them are single observations.

The labelling claim is the thesis and is unmeasured. The interface claim in §3, that labelled options cost one decision where unlabelled ones cost three readings, is what the whole selection chapter serves. There is no measurement of it here, not even a proxy: no reading time, no interaction count. A human study is the honest answer for whether the system helps someone write a better serial, but this narrower claim is testable for far less and was not tested.

7 Conclusion

The system described here is a bet that the durable part of AI-assisted long-form fiction is the state model rather than the generator. A chain of prompts is faster to build and easier to demonstrate; a state machine is what still works at episode forty, because it is the only one of the two that can answer what an edit invalidates.

Everything measurable about that bet is reported above, including where it does not yet pay. The deterministic tier is exact and free and says the mechanisms work: the checker catches what is injected into it, the dependency walk predicts a blast radius, the selector produces diverse shortlists. The live tier is four simulated writers, which is a measurement of the machinery and not of taste,

and it says the preference head recovers a meaningful fraction of what its estimator could recover on that much data. Whether any of this helps a human write a better serial is a question only a human study answers, and this document does not claim otherwise.

Three things would move the work furthest, in order. Measuring how often the model maintains the dependency graph it is asked to declare, which is currently the system's least-guarded assumption. Wiring the two learners that are built and measured but not yet on the writer's path. And running the evaluation against a writer who is not a linear functional over the head's own feature space.

References

Prithviraj Ammanabrolu, Wesley Cheung, Dan Tu, William Broniec, and Mark O. Riedl. Bringing stories alive: Generating interactive fiction worlds. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 16, pages 3–9, 2020.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.2307/2334029.

Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98)*, pages 335–336, Melbourne, Australia, 1998. ACM. doi: 10.1145/290941.291025.

Erin Cherry and Celine Latulipe. Quantifying the creativity support of digital tools through the creativity support index. *ACM Transactions on Computer-Human Interaction*, 21(4):21:1–21:25, 2014. doi: 10.1145/2617588.

Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4299–4307, 2017.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations (ICLR)*, 2021.

Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286, 2012. doi: 10.1561/2200000044.

Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022. doi: 10.1162/tacl_a_00453.

Piotr Mirowski, Kory W. Mathewson, Jaylen Pittman, and Richard Evans. Co-writing screenplays and theatre scripts with language models: Evaluation by industry professionals. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, 2023. doi: 10.1145/3544548.3581225.

Piotr Mirowski, Ben Wedin, Reinald Kim Amplayo, Rich Galt, Duncan Williams, Rida Qadri, Jaume Sanchez-Elias, Erin Drake-Kajioka, Sian Gooding, Lucia Lopez-Rivilla, Joao G. M. Araujo, Lion Schulz, Satinder Baveja, Shakir Mohamed, Edward Grefenstette, Laura Rimell, and Richard Evans. Fabula: Building a narrative storytelling sidekick with the writers' community, 2026.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 27730–27744, 2022.

Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. A tutorial on thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018. doi: 10.1561/2200000070.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018.

Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. Wordcraft: Story writing with large language models. In *Proceedings of the 27th International Conference on Intelligent User Interfaces (IUI '22)*, 2022. doi: 10.1145/3490099.3511105.

A Decisions, and the alternatives rejected

A.1 Free-tier keys for all of it, with the metered budget held in reserve

Rejected: Paying for a strong model from day one. Every number in this document was produced on free-tier keys: six rotated Gemini keys for generation and judging, Groq for extraction and classification, and CPU models on this machine for embeddings and natural language inference. **Total spend across 602 routed calls and 0.6 million tokens: $0.00.** The read-through cache served 50% of those calls. Embeddings and the NLI checkpoint run locally and bypass the router, so they are outside that token count, and outside nothing else, since they are free.

The number in that log that deserves a sentence is 290 errors against 602 calls. Almost all of them are free-tier rate limits, retried and served; the cost of a free tier is not paid in dollars but in wall-clock and in runs that stop early when a daily allowance is gone, which is visible in the run count above. A metered key removes that failure mode and adds a bill. Which trade is right depends on whether the thing being built is the system or the story, and during development it was the system.

That is a choice about where the risk is, not a constraint that was worked around. A metered key from day one buys better prose immediately and hides every cost bug until the bill arrives; a free tier forces the cost controls to exist before they are needed, and they are the same controls a production system needs: purpose-tag routing so extraction never touches the expensive model, a read-through cache keyed on the full request so a rerun of an identical configuration is nearly free, key rotation with per-(key, model) cooldowns, and a budget guard that refuses a call it cannot afford. Every one of those was exercised in anger rather than written and hoped for.

The metered budget existed for one purpose: the strong-model rerun, where the same evaluation runs against a better model to separate *the system works* from *the model is good*. That comparison is only meaningful because nothing was re-engineered in between. The model id is a configuration value and the routing is one table entry, so the rerun changes exactly one variable. §4.3 reports what it cost and what it showed.

A.2 Purpose-tag routing rather than per-call-site provider choice

The cost policy is six lines in one table. *Rejected:* Choosing a provider where the call is made. That scatters the policy across forty call sites and makes it impossible to audit or change. It also makes rerunning everything on a stronger model a one-line edit rather than a refactor: one routing table entry moves generation and judging to the metered provider and leaves extraction where it is, so the comparison changes one variable.

A.3 Diffs rather than replacement text

Every AI action is a typed operation list against the current state, not a block of prose that overwrites what was there. *Rejected:* Text-level replacement with a visual diff. It looks similar in the interface and is far weaker underneath: a text diff cannot say "this introduces a character", cannot be checked against the world graph before it is applied, and cannot tell the writer what it would invalidate.

A.4 Declared dependencies rather than inferred ones

A beat states what it produces and consumes; propagation walks those edges only. *Rejected:* Inferring dependencies with a model. A dependency oracle that is wrong ten percent of the time makes every propagation result untrustworthy, which is worse than no propagation at all, because the writer stops reading the marks. Embedding-similarity edges are implemented as a "maybe affected" signal and as an ablation to measure, never as the primary mechanism. That was a prediction about their

quality, and the measurement contradicts it: on the one case available they reach perfect precision and recall over a range of thresholds, where declared edges alone miss the undeclared dependency. The design decision still holds, because it is about what a writer can trust rather than about what scores well on one fixture, but the prediction was wrong and §4 says so.

A.5 A closed predicate vocabulary

10 predicates plus a free-text escape hatch. *Rejected:* Open-vocabulary relation extraction. Closed predicates make the first checker layer a dictionary lookup and an equality test: free, instant, and never wrong for the wrong reason. The escape hatch is exactly the set of facts that needs the more expensive NLI check, which is a useful way to have partitioned the problem.

A.6 Content-addressed snapshots

Rejected: Storing a full copy of the story per version, or an event log without materialized states. Full copies do not survive a few hundred snapshots. A pure event log makes "what is true right now" a replay, and makes branch comparison hard. Content addressing gives cheap versions, free structural sharing, and diff as set arithmetic.

A.7 Axis-conditioned sampling rather than a similarity penalty

Candidates are generated under named instructions and the name reaches the writer. *Rejected:* Sampling k times and reranking for dissimilarity. That is Fabula's approach, and it produces variants that differ without being *nameably* different. The reranker is still there, and it stops two candidates that came out similar anyway, but the axes are what make three options a decision rather than three readings.

A.8 A calibrated NLI threshold rather than a chosen one

0.35, measured against the local checkpoint, and swept in the evaluation. *Rejected:* A round number. The interesting case is a real contradiction the model is only moderately confident about (0.43), which a 0.5 threshold silently misses, and silent misses in a continuity checker are the failure that matters.

A.9 Marking rather than auto-repair

Rejected: Automatically regenerating stale downstream nodes. This is the single most tempting feature in the system and the one that would destroy it. It is precisely what Fabula's writers complained about, and the reason is not technical: a system that rewrites approved work without asking cannot be trusted with a manuscript.

A.10 13 interpretable features rather than a learned representation

Rejected: Embedding the candidate and letting the head learn its own features. With twenty comparisons that head would memorize rather than generalize, and, decisively, its weights would say nothing a writer could read. An interpretable feature vector is what lets the system explain its own ranking back to the person it is ranking for. Nine are fixed; the remaining 4 are one slot per writer-defined criterion, in creation order, zero when unfilled. A single averaged criterion score could not express that a writer weights *menace* twice as heavily as *warmth*, which is the one thing writer-defined criteria exist to say.

A.11 SQLite

Rejected: Postgres. One writer, one story, a few megabytes, and a file the author can copy is the actual workload. Postgres buys concurrency this system does not have. What would change at platform scale is discussed in the systems documentation.

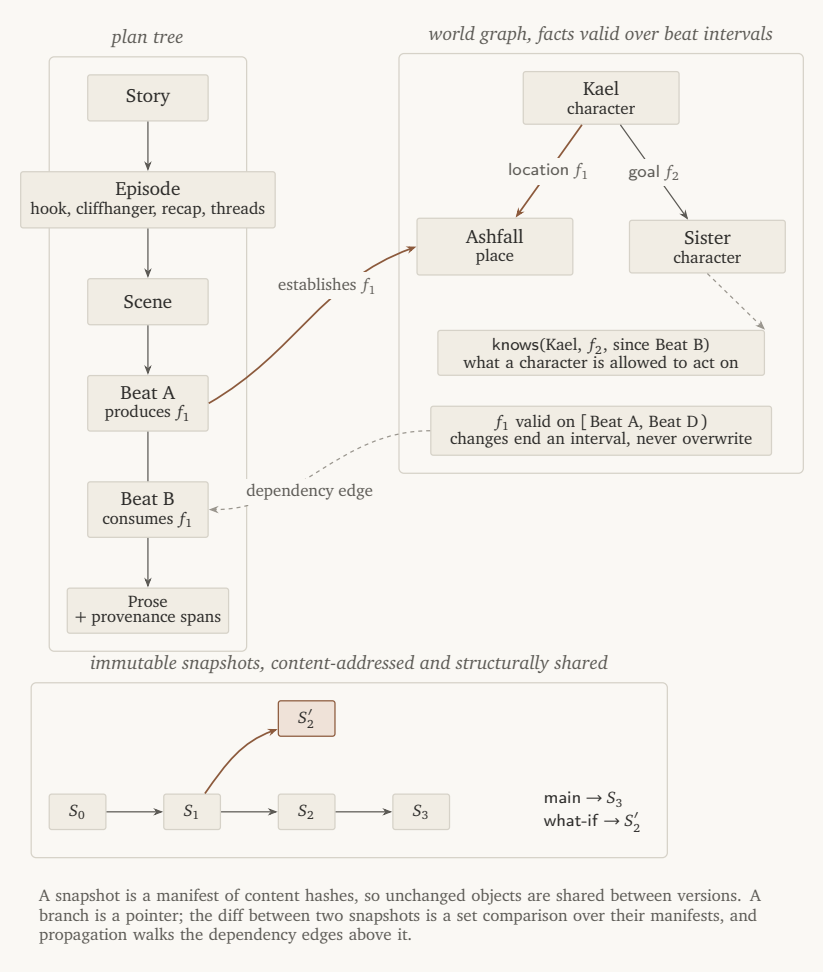


Figure 5: The three stores and how they reference each other.

B The writer ledger, and provenance

Provenance is recorded per sentence (human, ai, ai_edited_by_human), because the interesting case is mixed: the AI proposed a paragraph and the writer rewrote two sentences of it. A node-level flag would lose exactly the information the writer cares about, and the authorship ratio shown in the interface would be a lie.

C Components

Package	What it does
`domain/`	The typed state and the 31 diff operations that are the only way to change any of it. Frozen models, a pure `apply`, typed errors.
`store/`	Content-addressed objects, snapshot manifests, branch pointers, three-way merge. Git's data model, in one SQLite file.
`graph/`	Declared dependency edges, propagation that marks and never rewrites, and the entity-scoped slices generation prompts consume.
`providers/`	One interface over four backends, with key rotation, a read-through cache, a budget guard, and a call log. No key is ever logged.
`agents/`	Schema-constrained generation converted into typed diffs, plus fact extraction from accepted prose.
`selection/`	Named axes, MMR / DPP / top-k behind one signature, and the dial.
`continuity/`	The three checker layers, the bible diff, and the periodic audit.
`preference/`	Exemplars, edit mining, the voice model, the ranking head, the bandit.
`eval/`	Simulated writers, injected ground truth, metrics, ablations, the gallery recorder.
`api/` + `frontend/`	A thin HTTP adapter and a five-tab interface. No logic lives in either.

D The proposal engine in detail

D.1 Progressive levels

Generation is a function `propose(level, state slice, intent, k)`. The levels run seed → premise → episodes → scenes → beats → prose, and the writer settles each before the next exists. This is Fabula's progressive-build-up lesson taken literally: writers rejected systems that produced a whole plan before they had agreed to its premise.

Every level has a schema the model must fill, and the schema is where the shape of the answer lives. A beat proposal has fields for which facts it establishes and which existing facts it relies on, and for each new fact, the list of characters who *learn* it at this beat: not everyone present, and not everyone it is true of. That declaration is what the propagation walk later reads.

It is worth being exact about how strong that guarantee is, because the rest of the system leans on it. Those fields are *described and prompted for, not schema-required*: only the title and the body of a beat are mandatory, so a model that returns a beat and no dependencies validates, and a `consumes` id naming a fact that does not exist is dropped without a flag. The dependency graph is therefore as good as the model's compliance, and this system does not measure that compliance; see §6. The design decision behind the fields is still the right one: a declared edge is checkable and a guessed one is not. But "cannot go unmaintained" would be a claim about the model, not about the schema.

Every field carries a length bound, forwarded into the provider's own response schema where the provider honours one. This keeps candidates readable, and it closes a failure mode observed in the first live test: a small model fell into a repetition loop and filled a field until it hit the token cap, turning a good candidate into truncated JSON.

The bound is enforced client-side, and *how* is worth a sentence, because the obvious answer is wrong twice over. Rejecting an over-long field costs a repair call to recover a candidate that was fine except for being twenty characters long, measured at roughly 80% of proposals during the first evaluation run, which doubled the cost of the experiment. So the bound truncates. But truncating naively cuts mid-word, and the fragment becomes permanent state that every later prompt reads back as context:

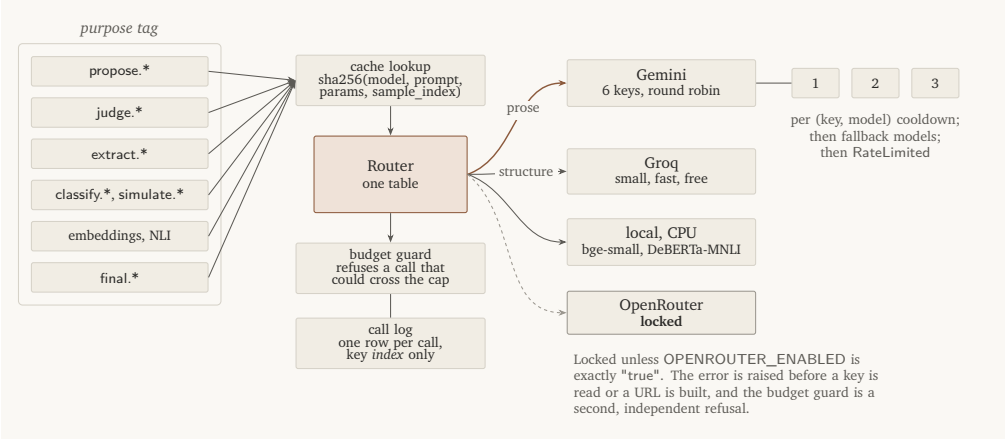


Figure 6: Purpose-tag routing, with rotation, caching, budget and logging around it.

about a third of one writer's candidates ended mid-clause, and one field arrived as literal keyboard mash padded to a minimum length and was committed. So the contract is neither: cut at the last sentence boundary inside the bound; ask again once when nothing inside the bound ends a sentence, or when the value is padding rather than writing; drop the candidate if the second answer is no better. A dropped candidate costs one sample out of six. A committed fragment costs every prompt after it.

D.2 Extraction, and the closed vocabulary

When prose is accepted, whether the model wrote it or the writer typed it, a cheap model reads it back and returns structured facts. A hand-written beat therefore participates in continuity exactly like a generated one: the system does not generate anything, it only reads what was written and updates the graph.

Predicates come from a closed vocabulary of ten, plus a free-text note. Closed is what makes the first checker layer a dictionary lookup and an equality test rather than a semantic judgement, and the escape hatch isolates exactly the residue that needs the more expensive check.

Names are canonicalized conservatively: exact match after normalization, otherwise a new entity *and* a note telling the writer what it might duplicate. No similarity threshold is used. A fuzzy match that is occasionally wrong would weld two characters together silently, which is the failure this whole system exists to prevent.

One routing table maps purpose tags to backends: prose and judging to Gemini, extraction and classification to Groq's small fast model, embeddings and NLI to local CPU models, and a metered provider that is locked. Around the chosen backend sit a read-through cache, a budget guard, and a call log that records tokens for every call.

Three details are load-bearing. Gemini's six free-tier keys carry per-`(key, model)` cooldowns, because free-tier quota is per model per key, and exhausting a key on one model says nothing about its allowance on another. The cache key includes a per-sample index, so the six axis-conditioned candidates of §3.6 are six entries rather than one. And non-prose purposes fall back across providers when one is down, while prose does not: a beat silently served by an 8B model is a quality regression the writer cannot see the cause of.

D.3 The generation loop

1. The writer states an intent at a node.
2. The engine slices the state, samples k candidates, converts each into a typed diff, and previews what each would invalidate.
3. The writer accepts, edits, or rejects. Editing keeps the writer's words and records the before/after pair; rejecting records the direction so it is not proposed again.
4. Accepting commits the diff, extracts facts from any new prose, and commits the propagation marks as a second snapshot.

5. Every action is recorded as a signal, along with the alternatives it was shown beside, which is what makes an accept a *preference* rather than an event.

E Continuity checking in detail

Layer 2: local NLI. The residue is the free-text note predicate and multi-valued facts whose object strings differ. A DeBERTa cross-encoder He et al. [2021] fine-tuned on MNLI scores each pair for contradiction. It is the same instrument, and used for the same reason, that the summarization-consistency literature uses to detect claims a source does not support Laban et al. [2022]. A cross-encoder rather than a bi-encoder because the distinction that matters here is exactly the one a bi-encoder structurally cannot make: "Kael is in Ashfall" and "Kael is not in Ashfall" embed almost identically, being about the same thing, but a model that reads both sentences together sees the negation.

The threshold is sanity-checked against cases whose answers are known rather than picked off a round number. On the local checkpoint a blunt contradiction scores near 1, a paraphrase near 0, and a real but universal-versus-specific contradiction near 0.43, so a 0.5 threshold would miss the third. The default is 0.35, and a regression test pins the ordering of those three cases so a checkpoint change that inverts them fails the suite. What this is not is a calibration curve: three sentence pairs establish that the model separates the cases, not where the boundary should sit, and no sweep of this threshold is reported. The soft-edge threshold in §4 is a different mechanism.

Layer 3: soft judge. Motivation plausibility and tone, plus the ranking score. It argues before it rates: the schema declares a sentence of reasoning ahead of each number, and field order is preserved, which is Fabula's own finding about their auto-rater and matches the general result that a score conditioned on an explanation is better calibrated. The honest version of that sentence is that the *ordering* is enforced where the provider honours schemas at all; the argument itself has a default and is not read back, so a model that skips it is not detected. It scores the writer's own criteria, in the writer's words, alongside fixed narratology axes.

E.1 What a flag must carry

Two properties are non-negotiable. **A deterministic flag cites its establishing beat.** "Kael cannot be in Ashfall here" is useless; "Kael was established in Kell in *The Warden's Offer*" is actionable. That holds for layers 1 and 2, where the flag is raised by a rule that knows which fact it read. Layer 3 is an opinion about a passage rather than a consequence of a fact, so it carries no such citation, and the interface does not pretend otherwise. And **nothing is ever auto-fixed**. A contradiction can be exactly what the writer meant: characters lie, narrators are unreliable, a flashback contradicts the present on purpose. A system that silently repairs those is worse than one that says nothing.

Hard flags (deterministic) and soft flags (opinions) are structurally distinct in the data model, sorted apart, and rendered differently.

E.2 The bible diff

On accept the writer sees what changed in the world, in sentences:

```
+ Kael keeps a secret:  he can hear the ash.
~ Kael is at Ashfall.  (no longer true from here)
- Kael has the Warden's token.
```

They can strike any line. Striking is not an undo. It produces a normal diff, deleting the fact if it was never true and ending it if it stopped being true, which propagates like any other change and re-runs layer 1 on whatever depended on it. Even correcting the machine goes through the same reviewable path as everything else.

E.3 The periodic audit

Per-accept checking is incremental and can miss slow drift: a contradiction assembled over ten episodes where no single accept was wrong. A periodic audit walks the whole graph through layers

1 and 2. It also answers a question no per-fact check can: which threads are being dropped. A thread opened in episode 2 and untouched since episode 3 is not a contradiction, so nothing else in the system would ever mention it, and for a serial it is the most expensive kind of mistake.

F The preference layer in detail

F.1 The signal is a comparison

Every accept is recorded with the alternatives that were on screen. "The writer liked A" is weak; "the writer preferred A to B and C, having seen all three" is a comparison, and comparisons are what Bradley-Terry is defined on. Pairs are formed only *within* one shown set, because comparing a candidate accepted on Monday against one rejected in a different scene on Friday would compare two different questions. An edited-then-accepted win is weaker evidence than a clean accept and is weighted accordingly.

F.2 The data regime

The head starts from a prior fitted across synthetic proto-personas and adapts per writer from there, because twenty comparisons cannot fit a model from scratch and there is no population of real writers to fit a prior on. The same caveat as the bandit applies and is worth stating in the same breath: the evaluation harness starts its heads from that prior; the product does not. A writer opening the application today starts at uniform weights. What makes the no-preference ablation exact is therefore not the prior's shape at zero data but an engine gate: below one recorded comparison the layer returns no opinion at all, and the ranker it would have influenced is bypassed.

That prior cannot contain real human taste. Nothing in its data ever met a reader. What it can contain is the structure of the problem: that features have consistent signs, that the signal is noisy, that some axes matter more than others. The bet is that the adaptation *machinery* transfers even though the values do not, and the bet is measured rather than asserted: a prior-initialized head reaches 0.780 held-out pairwise accuracy after a fresh writer's first ten decisions, against 0.700 from uniform initialization. Were that lift zero, the prior would be worthless and should be dropped.

There is a deeper circularity worth naming: the preference layer is validated against simulated writers made of the same feature vector the head is fitted on. That is why the evaluation measures *recoverability*, meaning whether the fitted weights correlate with the hidden ones, rather than claiming the system learns taste. Recoverability is a real property of the machinery. Taste is not something this evaluation can measure at all; only a human study would settle it.

F.3 Six learners, six data appetites

A writer produces about twenty comparisons in a session, which is nowhere near enough to fit one model that does everything. So each signal is exploited by the cheapest mechanism that can use it, and each contributes as soon as it has enough.

Learner	Needs	What it contributes
Exemplar retrieval	1 accept	The writer's own sentences in the prompt, and their rejected ones as negatives
Edit direction	1 edit	Mean embedding shift from the model's version to the writer's; score by projection onto it
Edit mining	2 similar edits	Plain-language style rules, into the visible ledger
Bradley-Terry head	$\sim$10 comparisons	The ranking
Voice model	$\sim$dozens of texts	"Sounds like this writer"
Thompson bandit	$\sim$20 shown sets	How much of the shown set to spend on exploration *(built and measured; not yet on the writer's path, see below)*

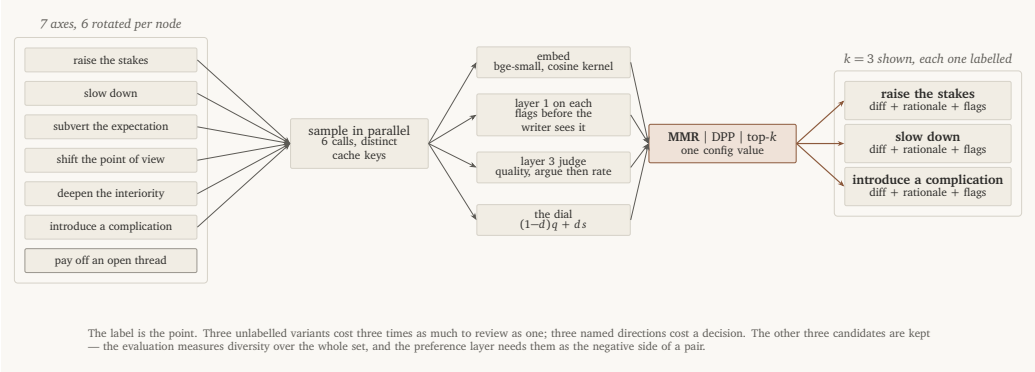


Figure 7: Axis-conditioned sampling and diverse selection.

F.4 The voice model

A judge can score whether a passage is well constructed; it cannot say whether it sounds like *this* writer, because that is an identity rather than a quality. A frozen bge-small encoder plus a 384×128 projection trained with InfoNCE van den Oord et al. [2018]: anchors are prose the writer wrote or edited, positives are what they accepted, negatives are what they rejected and the pre-edit versions of what they changed. Frozen encoder because a session's worth of prose would damage an encoder far more than it would personalize it; the projection only has to find which directions in an already-good space this writer cares about.

F.5 Exploration

A ranker that always shows its top three converges: the writer only ever sees what the model already believes, so the model never learns it was wrong. The answer built for this is two arms over the composition of the shown set, either three safe or two safe plus one high-surprise, chosen by Thompson sampling Russo et al. [2018]. Exploration anneals itself out of the posterior width, with no schedule to tune; ε-greedy would keep paying a fixed exploration cost forever and would explore uniformly, spending as much on an arm it is sure about as on a marginal one. One detail is worth the line: the arm is reported as *safe* whenever no unseen surprising candidate existed, so the bandit is never credited for exploration that did not happen.

It is not wired to the writer. The engine hands the writer the selector's top k; the bandit composes nothing on that path, its posterior never leaves $\mathrm{Beta}(1, 1)$, and the regret curve below is a simulator with the arms held at fixed rates. So the convergence problem this subsection opens with is stated, not solved, in the running system.

G Diversity and the dial in detail

Sampling three continuations from one prompt gives three phrasings of one idea, and the writer has no choice to make. Fabula's answer was to penalize similarity against previous suggestions, which produces variants that differ but are not *nameably* different, and their own finding is that unlabeled alternatives cost more to review than they save. Diversity in the embedding space is not diversity in the interface.

Axes. Seven named conditioning instructions: raise the stakes, slow down, subvert the expectation, shift the point of view, deepen the interiority, introduce a complication, and pay off an open thread. Six are rotated deterministically per node, sampled in parallel, and the label travels with the candidate to the screen.

Selection. MMR by default: $\lambda q_i - (1 - \lambda) \max_{j \in S} \mathrm{sim}(i, j)$, so the second pick is the best candidate that is not the first one again. A determinantal point process is implemented as a drop-in alternative: with kernel $L = \mathrm{diag}(q)\, S\, \mathrm{diag}(q)$, the determinant of a subset is the squared volume its quality-scaled vectors span, which is large when they are good *and* mutually dissimilar and zero when two are parallel. The diversity trade-off is the structure of the distribution rather than a tuned

parameter Kulesza and Taskar [2012], Carbonell and Goldstein [1998]. Plain top-k is the ablation baseline. All three share one signature, so the comparison is one config value.

The dial. Fabula's writers asked for an "absurdity dial". The request is deeper than it sounds: it asks to control the objective, not the output. A writer at 3am on episode 40 wants safe continuations; the same writer opening a new arc wants to be surprised. Take the greedy temperature-0 continuation as "what this model would obviously do next"; each candidate's surprise is its embedding distance from that; effective quality is $(1-d)\tilde{q} + d\tilde{s}$. At $d = 1$ the system is explicitly selecting against what the model would have done anyway. One extra cached call per node.

Candidates arrive already checked. Layer 1 runs on each candidate's would-be facts, by applying the diff to a scratch state and checking it there, so a candidate that contradicts the bible reaches the writer with the contradiction attached. Hard flags push a candidate down the ranking but never remove it.

G.1 Hard constraints, and what enforcing them cost

A hard constraint reaches the generation prompt *and* is checked at judge time: the judge receives the constraint list as explicit check items alongside the scoring axes, and returns the ones a candidate breaks, quoting each. Those become soft flags citing the constraint verbatim. Never blocked, since the writer decides here as with every other flag, but checked, which is the difference between a constraint and a wish. That is a change this system needed and did not have: a writer testing it added "Present day. No ten-shilling notes, no shillings, no Austin Cambridges" and was then offered four pounds ten, a Morris Minor and three-shilling bits. Their sentence for it is the one to keep: *a hard constraint that is not enforced at generation or at check time is a style note with a more confident name.* The check rides the judge call that already runs per candidate rather than adding one per constraint, which would multiply the most expensive stage of the pipeline by the size of the ledger.

H Evaluation design in detail

H.1 What the probe found, in detail

What the probe found, and what changed because of it. Several of the 13 features never varied inside a single writer's candidate sets. A constant column gives the fit no gradient, so those weights simply stayed wherever the population prior had put them. That is harmless in session, since a constant cannot reorder anything the writer sees, which is exactly why nothing had caught it, and wrong the moment the head is applied to candidates drawn from anywhere else. Only an off-distribution instrument could see it, and that is what the probe is.

Three things followed. Two of those features were dead for reasons that had nothing to do with the stories and everything to do with the code: dialogue ratio counted only double quotation marks while the model writes speech in single ones, so prose that was a third dialogue scored zero; and the voice model, which trains on prose the *writer* wrote or rewrote, had never once been trained, because no simulated writer had ever used the hand-write or polish paths the interface offers. Both are fixed and both features now move.

They do not move *much*, and the reason is the third thing, which is structural and is not a bug at all. The judge emits five levels. Four fixed axes and every writer-defined criterion are that judge's output rescaled, and with three candidates in a set the expected outcome is a tie: across the shipped probe fixture the two unfilled criterion slots are constant in every decision, two of the four judge axes are constant in more than half of them, and voice cosine, now that it varies at all, varies across a range too narrow to reorder anything at any weight the head can learn. That is the dominant cause of the constant columns, it was not fixed by fixing the two bugs, and it bounds how much a 13-dimensional head can be identified from a few dozen three-way comparisons at all.

The fourth finding is a property rather than a bug, and it changed the learner. A direction the data cannot identify should not inherit an opinion from a population prior, so fitting now shrinks unidentified directions towards zero instead. *Unidentified* here means exactly one thing and it is worth not overselling: a column that is constant across every comparison. Rank deficiency from two columns that always move together, which is the diagnosis the next paragraph gives for the writer who did worst, is invisible to this check and is not addressed by it. That was decided by an A/B

rather than by argument: both variants were refitted from the decisions already recorded and scored on the same held-out probe, which moved agreement from 0.412 to 0.468 and top-1 from 62% to 64% over 8 replayed runs, while the second half of the rule held: the prior's five-comparison lift came out at +0.067 with the change against +0.067 without it, inside the 0.040 spread across 5 seeds. That is a different measurement from the zero-comparison row in §4, which is negative: this one asks whether the change costs the prior anything where the prior helps, and the answer is that it does not. The rule the result had to clear was written down before the number existed, and the replay costs nothing, so there was no opportunity to rerun until a variant won. Each run still reports its own unexercised features in `summary.md`.

Why one writer did worst, in detail. The Minimalist was worst on every measure, and the recorded feature vectors say why. It is not "too little data", which was the obvious guess and is wrong: its candidate sets are *more* separated than the best performer's. Two of its three largest hidden weights fit with the wrong sign, and each has a cause that is a property of that writer rather than a defect in the machinery.

Its largest weight is on *specificity*, and the writer has also defined a criterion called *concreteness* which the judge scores $+0.73$ correlated with it, a near-duplicate column. Two collinear columns at a few dozen comparisons is a textbook small-sample regression: the fit hands the credit to one and compensates with a sign flip on the other, and here the fixed axis is the one that flips. Its second-largest weight is a large *negative* one on length, and its standing style note, "shorter sentences", is *followed*, so every candidate comes back at about the same length and the length feature carries a spread of 0.03. The head cannot learn a large negative weight from a column that does not vary.

That second mechanism generalises and is worth stating: **a writer whose instruction is obeyed leaves the preference layer nothing to learn on the same axis**. The prompt side and the learning side are two halves of one system competing for the same signal, and this run is what it looks like when the prompt side wins. Neither is patched: rewriting the persona's criteria to be less collinear would be tuning the experiment to flatter the estimator.

The acceptance trend. Acceptance held or rose in three runs and fell in one (100% to 60%). Read on its own this says less than it appears to: a persona's bar is a fixed quantile of its own score distribution and does not move, while each new candidate has to stay consistent with more established facts, more open threads, and more accumulated rules, so the curve tracks task difficulty as well as satisfaction, and the probe above is the view that does not. The thirds also rest on a handful of decisions each, at 4 decisions per run, so the endpoints are noisy.

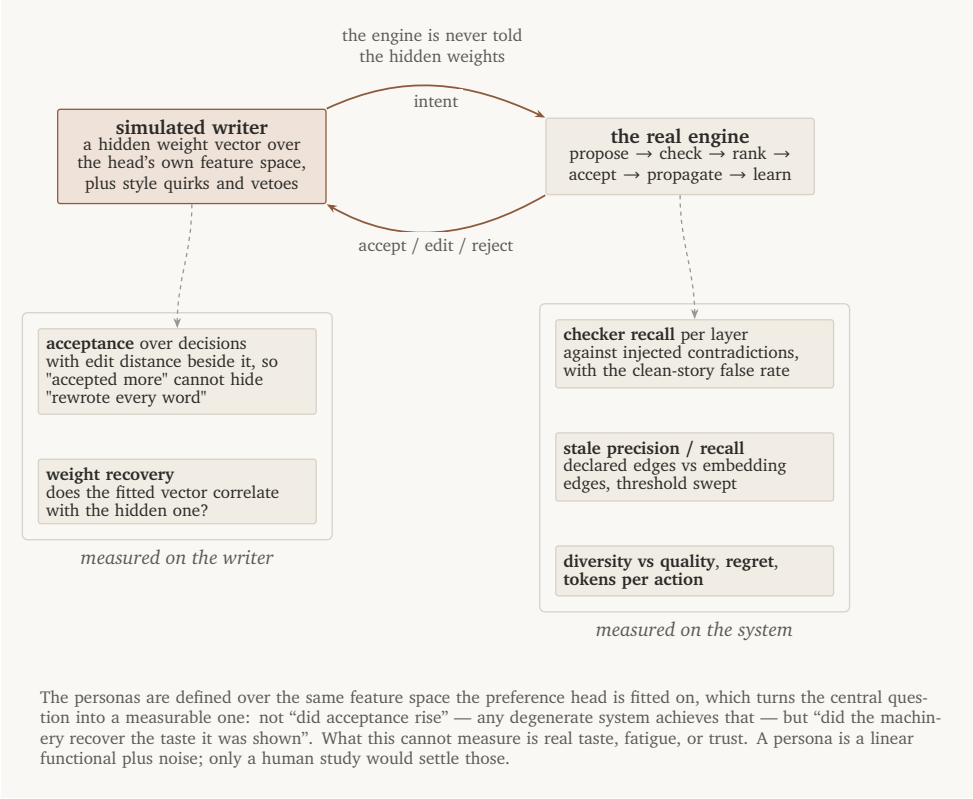
Curves per persona are in `eval/results/`, and the full table of probe readings, ceilings, retry rescues, per-run costs, and anything that did not run is in `summary.md`. The earlier run, made before this pass changed the feature space and the harness, is archived under `eval/results/archive/`. Every number in this section regenerates with

```
python -m eval.run --config full && python -m eval.texnumbers
```

Everything above ran on **free-tier keys**: six rotated Gemini keys, Groq for extraction, and CPU models on this machine for embeddings and NLI, at a total spend of **$0.00** across 0.6 million of routed tokens. Prose quality is therefore a floor rather than a ceiling, and that is deliberate: the metered budget is held in reserve for the strong-model rerun, which is the run that separates "the system works" from "the model is good". It is a one-line configuration change, costed against the cap before it is spent.

H.2 Simulated writers

Four personas, each **a hidden weight vector over exactly the feature space the preference head is fitted on**, plus style quirks and forbidden moves, which are absolutes a linear preference cannot express and which real writers have.



The personas are defined over the same feature space the preference head is fitted on, which turns the central question into a measurable one: not "did acceptance rise" — any degenerate system achieves that — but "did the machinery recover the taste it was shown". What this cannot measure is real taste, fatigue, or trust. A persona is a linear functional plus noise; only a human study would settle those.

Figure 8: The evaluation loop and where each measurement is taken.

Persona	Weighted towards	Character
the Serialist	momentum, consequence	Writes for retention; no flashbacks, they stall
the Minimalist	specificity, *negative* on length	Short, concrete, exposition-averse
the Maximalist	voice, interiority, length	Dial at 0.70; dialogue low
the Controller	continuity, own criteria	Accepts rarely, edits heavily, locks constantly

Defining the personas over the head's own feature space is what makes the central question measurable: not *did acceptance rise*, which a degenerate system achieves by proposing the same safe thing repeatedly, but *did the fitted weight vector correlate with the hidden one*. Three constraints keep it from being circular, the first asserted in a test: the engine is never told the hidden weights; the persona sees only what the engine chose to show; and the model that performs the persona's edits is told their *style*, never their objective.

H.3 Ground truth by construction

Recall needs a denominator. One contradiction of each class is injected into its own story, built fresh from the same clean scenario, and each is scored on its own, because injected together they interact: two of the five land on the same beat and a third kills a character the others need alive, and a catch is credited when a flag names any of an injection's facts, so shared-story cross-talk would score a location conflict as caught because the possession conflict beside it was. That isolation was added after this review found the fixture had been sharing one story; the reported numbers did not move, which is the useful outcome: the defect was real and the measurement survived it. Extraction is scored against hand-written passages with known fact sets. Staleness is scored against a scripted edit whose true blast radius is known because the dependency edges were written by hand, and which deliberately includes a beat that *depends on the changed fact but never declared it*. Otherwise

the soft-edge ablation could only add false positives, and that would be an unfair test of an idea worth testing properly.

I Reproduction

Every number in this document is a macro generated from the evaluation's own output. The deterministic tier needs no network and no keys:

```
python -m eval.offline        # every exact metric, plus the figures
python -m eval.texnumbers     # emit docs/results.tex
tectonic docs/presentable.tex
```

The live tier needs provider keys in a workspace `.env` and takes a few hours on a free tier, because it waits out rate limits rather than working around them:

```
python -m eval.run --config full --max-calls 2000
python -m eval.texnumbers
```

`scripts/rerun_until_quota.sh` wraps that call for an unattended run: it probes for capacity rather than sleeping until a clock time, and relaunches on failure, which resumes from the per-episode checkpoints rather than restarting. The suite, the linters and the type checker run with `make check`; `scripts/e2e\_smoke.sh` drives the whole stack over HTTP against a mock provider, so it needs neither network nor quota.